

Aula React Native + TanStack Query & PokeAPI

Esta aula demonstra como buscar dados de uma API REST usando o TanStack Query (@tanstack/react-query) e exibi-los em uma lista com FlatList.

Importacoes

Os componentes nativos utilizados sao: **ActivityIndicator** para exibir o carregamento, **FlatList** para renderizar a lista de pokemons e **Text** e **View** para estrutura e texto. O **SafeAreaView** garante que o conteudo respeite as areas seguras do dispositivo (notch, barra de status). Do TanStack Query sao importados: **QueryClient**, **QueryClientProvider** e o hook **useQuery**.

```
import React from 'react'
import { ActivityIndicator, FlatList, Text, View } from 'react-native'
import { SafeAreaView } from 'react-native-safe-area-context'
import {
  QueryClient,
  QueryClientProvider,
  useQuery,
} from '@tanstack/react-query'
```

Tipagem com TypeScript

O tipo **Pokemon** descreve o formato de cada item retornado pela PokeAPI. Cada pokemon possui um **name** (nome) e uma **url** com o endpoint de detalhes daquele pokemon.

```
type Pokemon = {
  name: string
  url: string
}
```

QueryClient e QueryClientProvider

O **QueryClient** e a instancia central do TanStack Query. Ele gerencia o cache, o estado das requisicoes e as configuracoes globais. O **QueryClientProvider** disponibiliza essa instancia para todos os componentes filhos via contexto do React.

```
const queryClient = new QueryClient()

export default function Index() {
  return (
    <QueryClientProvider client={queryClient}>
      <SafeAreaView style={{ flex: 1 }}>
        <PokemonList />
      </SafeAreaView>
    </QueryClientProvider>
  )
}
```

Sem o **QueryClientProvider** envolvendo a arvore de componentes, o hook **useQuery** nao funciona e lancara um erro em tempo de execucao.

Funcao de busca: fetchPokemons

A funcao **fetchPokemons** e uma funcao assincrona que faz uma requisicao GET para a PokeAPI. O parametro **limit=20** limita o retorno a 20 pokemons. Se a resposta nao for bem-sucedida, um erro e lancado. Caso contrario, retorna apenas o array **results** do JSON.

```
async function fetchPokemons() {
  const response = await fetch(
    'https://pokeapi.co/api/v2/pokemon?limit=20'
  )

  if (!response.ok) {
    throw new Error('Erro ao buscar pokemons')
  }

  const data = await response.json()
  return data.results
}
```

Parte	Descricao
fetch(url)	Faz a requisicao HTTP GET para a URL informada
response.ok	True se o status HTTP for entre 200-299
response.json()	Converte o corpo da resposta de JSON para objeto JS
data.results	Array de pokemons retornado pela PokeAPI

Hook useQuery

O **useQuery** e o hook principal do TanStack Query. Ele executa a funcao de busca automaticamente, gerencia o cache e expoe o estado da requisicao atraves das propriedades abaixo.

```
const {
  data,
  isLoading,
  error,
} = useQuery({
  queryKey: ['pokemons'],
  queryFn: fetchPokemons,
})
```

Propriedade	Tipo	Descricao
data	Pokemon[] undefined	Dados retornados pela queryFn apos sucesso
isLoading	boolean	True enquanto a primeira busca esta em andamento
error	Error null	Objeto de erro caso a requisicao falhe
queryKey	unknown[]	Chave unica que identifica e cacheia esta query

queryFn	() => Promise	Funcao que realiza a busca dos dados
---------	---------------	--------------------------------------

Tratamento de estados da requisicao

O componente **PokemonList** trata tres estados possiveis: carregando, erro e sucesso. Cada estado renderiza uma interface diferente.

```
// Estado de carregamento
if (isLoading) {
  return (
    <SafeAreaView>
      <ActivityIndicator size="large" />
    </SafeAreaView>
  )
}

// Estado de erro
if (error) {
  return (
    <SafeAreaView>
      <Text>Erro ao carregar pokemons</Text>
    </SafeAreaView>
  )
}
```

Estado	Condicao	Interface exibida
Carregando	isLoading === true	Spinner centralizado com ActivityIndicator
Erro	error !== null	Mensagem de erro em texto simples
Sucesso	data disponivel	FlatList com a lista de pokemons

Renderizando a lista com FlatList

O **FlatList** e otimizado para listas longas: renderiza apenas os itens visiveis na tela. O **keyExtractor** usa o nome do pokemon como chave unica. O **renderItem** define o layout de cada item. O **textTransform: 'capitalize'** formata o nome com letra maiuscula.

```

<FlatList
  data={data}
  keyExtractor={(item: Pokemon) => item.name}
  contentContainerStyle={{ padding: 16 }}
  renderItem={({ item }) => (
    <View
      style={{
        padding: 16,
        marginBottom: 12,
        borderWidth: 1,
        borderRadius: 8,
      }}
    >
      <Text style={{ fontSize: 18, textTransform: 'capitalize' }}>
        {item.name}
      </Text>
    </View>
  )}
/>

```

Prop	Descricao
data	Array de itens a ser renderizado
keyExtractor	Funcao que retorna uma chave unica por item
contentContainerStyle	Estilo aplicado ao container interno da lista
renderItem	Funcao que define o componente de cada item

Resumo

QueryClient gerencia o cache e as configuracoes globais do TanStack Query. **QueryClientProvider** disponibiliza o client para toda a arvore de componentes. **useQuery** executa a busca automaticamente e expoe **data**, **isLoading** e **error**. **queryKey** identifica e cacheia cada query de forma unica. **FlatList** renderiza listas longas de forma performatica. **ActivityIndicator** exhibe o spinner durante o carregamento.